
🔴 **SAVOL 26:**

QAROR DARAXTI (`DECISION TREE`) ALGORITMI: TUGUNLAR, SHOXLAR VA BARGLAR. `GINI IMPURITY` VA `INFORMATION GAIN` KRITERIYALARI. DARAXTNING CHUQURLIGI VA `OVERFITTING` MUNOSABATI.

---

1. Qaror Daraxti — Intuitiv Tushuncha

Qaror daraxti — insonning **"Agar ... bo'lsa, unda ..."** mantiqini matematiklashtirgan algoritm. Har bir qaror — savol, har savol ma'lumotni ikki yoki undan ortiq guruhga bo'ladi.

```
Analogiya — Tabib tashxisi:
  "Harorat 38°C dan yuqorimi?"
        ↙ Ha          ↘ Yo'q
  "Yo'tal bormi?"     "Bosh og'riqmi?"
    ↙ Ha  ↘ Yo'q        ↙ Ha  ↘ Yo'q
  Gripp  Angina    Migren  Sog'

Bu aynan Qaror Daraxti!
```

Terminologiya:

```
Ildiz tugun (Root Node):
  → Eng yuqoridagi tugun
  → Butun ma'lumotni o'z ichiga oladi
  → Birinchi va eng muhim savol

Ichki tugun (Internal Node):
  → O'rta qismdagi tugunlar
  → Savol beradi → Ma'lumotni bo'ladi

Shox (Branch):
  → Tugunlar orasidagi bog'lanish
  → Savolning "Ha" yoki "Yo'q" javobi

Barg (Leaf Node):
  → Eng pastdagi tugunlar
  → Savol bermaydigan, javob qaytaradigan tugun
  → Klassifikatsiyada: sinf (0/1, spam/ham)
  → Regressiyada: o'rtacha qiymat

Chuqurlik (Depth):
  → Ildizdan bargga eng uzun yo'l
  → Chuqurlik ↑ → Murakkablik ↑ → Overfitting xavfi ↑
```

---

2. Daraxt Qanday O'rganadi? — Rekursiv Bo'lish

Qaror daraxti **greedy (ochko'z)** algoritm — har qadamda eng yaxshi bo'lishni tanlaydi, kelajakka qaramaydi.

```
Algoritm (CART — Classification And Regression Trees):

1. Barcha o'zgaruvchilarning barcha mumkin bo'lgan
   bo'lish nuqtalarini ko'rib chiqish

2. Har bo'lish uchun "Sifat" ni hisoblash
   (Gini Impurity yoki Information Gain)

3. Eng yaxshi bo'lishni tanlash

4. Ma'lumotni ikki guruhga bo'lish

5. Har guruh uchun 1-4 ni rekursiv takrorlash

6. To'xtatish sharti bajarilguncha:
   → max_depth yetildi
   → Tugunida min_samples_split dan kam nuqta
   → Tozalik (purity) yetarli
```

Masalan: 4 ta o'quvchi, 2 ta belgi:

```
Ma'lumot:
  O'quvchi | Soat (X₁) | Uxlash (X₂) | Natija (Y)
  ─────────|──────────|──────────────|──────────
  Ali      |     8    |    Yaxshi    |   O'tdi ✅
  Bobur    |     2    |    Yaxshi    |  O'tmadi ❌
  Camila   |     7    |    Yomon     |   O'tdi ✅
  Dilnoza  |     1    |    Yomon     |  O'tmadi ❌

Savol 1: "Soat > 5?"
  Ha: [Ali✅, Camila✅] → Hammasi O'tdi → Toza barg!
  Yo'q: [Bobur❌, Dilnoza❌] → Hammasi O'tmadi → Toza barg!

Daraxt:
        [Soat > 5?]
        ↙ Ha    ↘ Yo'q
    [O'tdi✅] [O'tmadi❌]
```

---

3. Gini Impurity — Nazariy Asos

Gini Impurity — tugunning "qanchalik aralash" ekanini o'lchaydigan metrika. CART algoritmida ishlatiladi.

```
Formula:
  Gini(t) = 1 - Σ pᵢ²
                i

  pᵢ → i-chi sinf ulushi (nisbati)

Talqin:
```

```
  Gini = 0.0  → Mukammal toza (faqat 1 sinf) → Barg bo'ladi
  Gini = 0.5  → Eng aralash (ikkilik sinfda: 50/50)
  Gini = 1.0  → Nazariy maksimal (ko'p sinf)
```

Hisoblash misoli:

```
Tugun A: 10 ta O'tdi, 0 ta O'tmadi
  p₁ = 10/10 = 1.0,  p₂ = 0/10 = 0.0
  Gini(A) = 1 - (1.0² + 0.0²) = 1 - 1 = 0.0
  → Mukammal toza! ✅

Tugun B: 5 ta O'tdi, 5 ta O'tmadi
  p₁ = 5/10 = 0.5,  p₂ = 5/10 = 0.5
  Gini(B) = 1 - (0.5² + 0.5²) = 1 - 0.5 = 0.5
  → Eng aralash ❌

Tugun C: 7 ta O'tdi, 3 ta O'tmadi
  p₁ = 0.7,  p₂ = 0.3
  Gini(C) = 1 - (0.7² + 0.3²) = 1 - (0.49 + 0.09) = 0.42
```

Bo'lish sifatini hisoblash:

```
Bo'lishdan oldin Gini hisoblanadi,
Bo'lishdan keyin o'rtacha (vazn bilan) Gini hisoblanadi:

Gini_split = (n_chap/n_jami)·Gini(chap) + (n_o'ng/n_jami)·Gini(o'ng)

Eng yaxshi bo'lish → Gini_split MINIMAL bo'lgan bo'lish

Gini Gain = Gini_oldin - Gini_split
             → Katta bo'lsa → Bo'lish yaxshi
```

---

4. Information Gain — Entropy Asosida

Entropy — ma'lumot nazariyasidan olingan tushuncha (Claude Shannon, 1948). Tizimning "tartibsizligini" o'lchaydi. ID3 va C4.5 algoritmlarida ishlatiladi.

```
Entropy formulasi:
  H(t) = -Σ pᵢ · log₂(pᵢ)
           i

  log₂ — 2 asosli logarifm (bit)

  H = 0.0  → Mukammal toza (log₂(1) = 0)
  H = 1.0  → Eng aralash (ikkilik: 50/50)
```

Hisoblash misoli:

```
Tugun: 5 ta O'tdi, 5 ta O'tmadi
  H = -(0.5·log₂(0.5) + 0.5·log₂(0.5))
  H = -(0.5·(-1) + 0.5·(-1))
  H = -(-0.5 - 0.5) = 1.0 bit
```

```
  → Maksimal tartibsizlik

Tugun: 8 ta O'tdi, 2 ta O'tmadi
  H = -(0.8·log₂(0.8) + 0.2·log₂(0.2))
  H = -(0.8·(-0.322) + 0.2·(-2.322))
  H = -(−0.258 − 0.464) = 0.722 bit
```

Information Gain:

```
IG(T, bo'lish) = H(ota) - Σ (nᵢ/n)·H(bola_i)
                           i

Katta IG → Bo'lish ko'p "ma'lumot" beradi → Yaxshi bo'lish

Misol:
  Ota tugun: [5 O'tdi, 5 O'tmadi] → H = 1.0

  Bo'lish A: "Soat > 5?"
    Chap [4 O'tdi, 1 O'tmadi]: H = 0.722
    O'ng [1 O'tdi, 4 O'tmadi]: H = 0.722
    IG(A) = 1.0 - (5/10·0.722 + 5/10·0.722) = 0.278

  Bo'lish B: "Soat > 3?"
    Chap [4 O'tdi, 0 O'tmadi]: H = 0.0
    O'ng [1 O'tdi, 5 O'tmadi]: H = 0.65
    IG(B) = 1.0 - (4/10·0.0 + 6/10·0.65) = 0.61

  IG(B) > IG(A) → B bo'lish afzal!
```

---

5. Gini vs Entropy — Qaysi Birini Tanlash?

```
Xususiyat        | Gini Impurity    | Entropy
─────────────────|──────────────────|──────────────────
Formula          | 1 - Σpᵢ²         | -Σpᵢ·log₂(pᵢ)
Hisoblash        | Tezroq (log yo'q)| Sekinroq
Natija           | Deyarli bir xil  | Deyarli bir xil
sklearn standarti| criterion="gini" | criterion="entropy"
Farqi            | ~1-2% farq       | Amalda muhim emas

Xulosa: Amalda ikkalasi deyarli bir xil natija beradi.
        Gini biroz tezroq → Ko'p ishlatiladi.
        Katta ma'lumotda tezlik muhim bo'lsa → Gini.
```

---

6. Chuqurlik va Overfitting Munosabati

Bu savol **eng muhim nazariy tushuncha:**

```
Chuqurlik = 1 (Faqat bitta savol):
  Juda sodda → Underfitting
  Ma'lumotni qo'pol bo'ladi
```

```
Chuqurlik = 2-5:
  Optimal hudud
  Asosiy naqshlarni ushlaydi

Chuqurlik = cheksiz (max_depth=None):
  Har barg faqat 1 ta nuqtani o'z ichiga oladi
  Train Accuracy = 100%  ← Yodlagan!
  Test Accuracy = Past   ← Overfitting!
```

Nima uchun chuqur daraxt yodlaydi?

```
Har bo'lish → Ma'lumot kichik guruhlarga bo'linadi
Oxirgi barg → 1-2 ta nuqta → Bu nuqtalarning o'ziga xos
              xususiyatlarini "qoida" sifatida yozadi
              (shovqinni ham!)

Yangi nuqta kelganda:
  → Shu "qoida" boshqa nuqtalarga mos kelmaydi
  → Bashorat yomon
```

Vizual tushuncha:

```
Oddiy daraxt (depth=2):          Chuqur daraxt (depth=10):


  ____                               _____
 | X>5|                             |X>5.1|
 /    \                            /      \
[0]  [1]                        [X>2.3]  [X>8.7]
                                /    \   /    \
2 ta qaror →                   Ko'p qaror → Har nuqta
Umumiy naqsh                   uchun alohida "qoida"
```

---

7. Regularizatsiya — Daraxtni Cheklash

```
Giperparametr    | Ta'siri
─────────────────|──────────────────────────────────────
max_depth        | Maksimal chuqurlik. Eng muhim!
                 | Kichik → Underfitting, Katta → Overfitting

min_samples_split| Bo'lish uchun minimal nuqtalar soni
                 | Katta → Kam bo'lish → Sodda daraxt

min_samples_leaf | Bargda minimal nuqtalar soni
                 | Katta → Umumlashtirilgan barglar

max_features     | Bo'lishda ko'riladigan max o'zgaruvchi
                 | Kichik → Tasodifiylik ↑, Overfitting ↓

max_leaf_nodes   | Maksimal barg soni
                 | Bevosita daraxt o'lchamini cheklaydi
```

Pruning (Budash) — Overfitting dan Himoya:

```
Pre-pruning (oldindan budash):
  → O'rgatish paytida yuqoridagi giperparametrlar
  → "Qachon to'xtatish kerak?" → Oldindan belgilangan

Post-pruning (keyindan budash):
  → Avval to'liq daraxt o'stiriladi
  → Keyin "foydali bo'lmagan" shoxlar kesiladi
  → Cost Complexity Pruning (sklearn da: ccp_alpha)

ccp_alpha:
  → 0: Budash yo'q (to'liq daraxt)
  → Katta: Ko'p budash (sodda daraxt)
  → Optimal qiymat → Cross-validation bilan topiladi
```

---

8. Qaror Daraxti ning Afzalliklari va Kamchiliklari

```
Afzalliklari:
  ✅ Tushunish oson — grafik chizish mumkin
  ✅ "Nima uchun?" ga javob bera oladi (interpretable)
  ✅ Standartlashtirish shart emas
  ✅ Kategoriyali va raqamli ma'lumot uchun mos
  ✅ Muhim o'zgaruvchilarni o'zi aniqlaydi
  ✅ Outlierga nisbatan bardoshli

Kamchiliklari:
  ❌ Overfitting ga moyil (chuqur daraxt)
  ❌ Beqaror — ma'lumot bir oz o'zgarse, daraxt keskin o'zgaradi
  ❌ Greedy algoritm — global optimal emas, lokal optimal
  ❌ XOR kabi murakkab munosabatlarni tushuntirolmaydi
  ❌ Chiziqli munosabatni ko'p qadamda ifodalaydi
```

Beqarorlik muammosi:

```
Train ma'lumotiga 2 ta yangi nuqta qo'shilsa →
Daraxt tuzilishi butunlay o'zgarishi mumkin!

Yechim → Random Forest:
  Ko'p daraxt + Ovoz berish → Barqaror, aniq
  (Bir daraxt yomon, 100 ta daraxt kuchli!)
```

---

9. Xulosa

```
Qaror Daraxti:
  Ildiz → Ichki tugun → Barg
  Har tugun: Savol → Ma'lumotni ikki guruhga bo'ladi
  Barg: Yakuniy javob (sinf yoki qiymat)

Bo'lish Kriteryalari:
  Gini    = 1 - Σpᵢ²          → Tezroq, sklearn standarti
  Entropy= -Σpᵢ·log₂(pᵢ)      → Biroz sekin, deyarli bir xil natija
  Maqsad: Eng toza bo'lishni topish (Gini minimal, IG maksimal)
```

```
Chuqurlik va Overfitting:
  Chuqurlik ↑ → Murakkablik ↑ → Overfitting ↑
  max_depth cheklash → Eng muhim regularizatsiya
  Optimal depth → Cross-validation bilan topiladi

Overfitting belgilari:
  Train Accuracy >> Test Accuracy
  Bargda 1-2 ta nuqta

Oldini olish:
  max_depth, min_samples_leaf, min_samples_split
  Post-pruning (ccp_alpha)
  → Yoki: Random Forest ishlatish
```

> 💡 **Eslab qol:** Qaror daraxti — kuchli, lekin **yolg'iz ishlatganda beqaror** algoritm. Shuning uchun amalda Random Forest (ko'p daraxt) yoki Gradient Boosting ishlatiladi. Lekin ularni tushunish uchun avval **bitta daraxtni** mukammal tushunish kerak — bu poydevor!

🔴 **SAVOL 27:**

`RANDOM FOREST` ALGORITMINING `DECISION TREE` DAN USTUNLIGI: `BAGGING` PRINSIPI, KO'P DARAXTLAR VA O'RTACHA OVOZ BERISH. `FEATURE IMPORTANCE` KO'RSATKICHI NIMA VA QANDAY ISHLATILADI?

---

1. Asosiy G'oya — Jamoa Donoligi

Random Forest ning falsafasi bir jumla bilan:

```
"Ko'p mustaqil ekspertning o'rtacha fikri —
 bitta dahoning fikridan ko'ra ishonchliroq"

Misol:
  Bitta tabib tashxisi → 70% to'g'ri
  100 ta mustaqil tabib ovoz beradi → 95% to'g'ri

Nima uchun?
  Har tabib o'zining xatosini qiladi
  Lekin TURLI xatolarni → O'rtacha olganda xatolar bekor bo'ladi
  Umumiy to'g'rilik oshadi

Random Forest ham xuddi shunday:
  Bitta Decision Tree → Beqaror, overfitting
  100 ta mustaqil Decision Tree → Barqaror, aniq
```

---

2. Decision Tree ning Muammolari — Nima uchun RF Kerak?

```
Muammo 1 — Yuqori Variance (Beqarorlik):
  Train ma'lumotiga 5 ta yangi nuqta qo'shilsa →
  Daraxt tuzilishi butunlay o'zgarishi mumkin!
  → Bashorat keskin farq qiladi

Muammo 2 — Overfitting:
  Chuqur daraxt → Train=99%, Test=65%
  → Yodlagan, o'rganmagan

Muammo 3 — Greedy tanlov:
  Har bo'lishda lokal optimal → Global optimal emas
  → Muhim o'zgaruvchi e'tibordan chetda qolishi mumkin

Random Forest bularning barchasini hal qiladi!
```

---

3. Bagging — Asosiy Princip

Bagging = **B**ootstrap **Agg**regat**ing**

Ikki qismdan iborat:

Bootstrap — Ma'lumotni Tasodifiy Namunalash:

```
Asl ma'lumot: [A, B, C, D, E, F, G, H, I, J]  (10 ta)

Bootstrap namuna 1: [A, C, C, F, A, H, J, B, E, G]  ← Qayta tanlanishi
mumkin!
Bootstrap namuna 2: [B, D, A, I, F, F, C, H, E, J]
Bootstrap namuna 3: [G, A, J, B, D, C, F, I, A, H]
...
Bootstrap namuna N: [D, E, B, G, C, J, A, F, H, D]

Muhim: Bir xil nuqta bir necha marta tanlanishi mumkin!
       Ba'zi nuqtalar umuman tanlanmasligi mumkin (Out-of-Bag)

Statistik:
  Har bootstrap namunasida asl ma'lumotning ~63.2% i keladi
  ~36.8% i kelmaydi → Bu "Out-of-Bag" (OOB) namunasi →
  Bepul validatsiya to'plami!
```

Nima uchun 63.2%?

```
n ta elementdan n marta tasodifiy olish (qayta qo'yib):
Bitta element tanlanmaslik ehtimoli: (1 - 1/n)ⁿ

n → ∞:  lim(1 - 1/n)ⁿ = 1/e ≈ 0.368

Demak tanlanish ehtimoli: 1 - 0.368 = 0.632 = 63.2%
```

Aggregating — Natijalarni Birlashtirish:

```
Klassifikatsiya (Ovoz berish):
  Daraxt 1: Sinf A
```

```
  Daraxt 2: Sinf B
  Daraxt 3: Sinf A
  Daraxt 4: Sinf A
  Daraxt 5: Sinf B
  ──────────────────
  Ovoz: A=3, B=2 → Yakuniy: Sinf A ✅

Regressiya (O'rtacha):
  Daraxt 1: 245,000
  Daraxt 2: 267,000
  Daraxt 3: 251,000
  Daraxt 4: 238,000
  Daraxt 5: 259,000
  ──────────────────
  O'rtacha: 252,000 → Yakuniy bashorat
```

---

4. Random Forest = Bagging + Feature Randomness

Random Forest oddiy Bagging dan **yana bir muhim farq** bilan ajralib turadi:

```
Oddiy Bagging:
  Har daraxt → Tasodifiy bootstrap namuna
  Har bo'lishda → BARCHA o'zgaruvchilar ko'rib chiqiladi
  Muammo: Agar 1 ta o'zgaruvchi juda kuchli bo'lsa →
          Barcha daraxtlar unga o'xshash bo'ladi →
          Daraxtlar o'zaro korrelyatsiyali → Yaxshilanish kam

Random Forest:
  Har daraxt → Tasodifiy bootstrap namuna (Bagging)
  Har bo'lishda → Faqat m ta TASODIFIY o'zgaruvchi ko'riladi!
  m = √p (klassifikatsiya) yoki p/3 (regressiya)
  p = jami o'zgaruvchilar soni

Natija:
  Daraxtlar bir-biridan MUSTAQIL → Korrelyatsiya past →
  O'rtalashtirish ko'proq foyda beradi!
```

Bu muhim g'oya:

```
Kuchli o'zgaruvchi bor deylik (masalan: "Yosh")
Barcha daraxtlar "Yosh" dan boshlansa →
  Daraxtlar bir-biriga o'xshash → O'rtalashtirish foydasiz

Feature randomness:
  Ba'zi daraxtlarda "Yosh" tanlanmaydi →
  Boshqa o'zgaruvchilar ham imkoniyat oladi →
  Daraxtlar TURLICHA → O'rtalashtirish samarali!
```

---

5. Variance Kamayishi — Matematik Asos

```
Bitta daraxt variansi: σ²

N ta mustaqil (korrelyatsiyasiz) daraxt o'rtachasi variansi:
  Var(o'rtacha) = σ²/N

N=100 daraxt:
  Var = σ²/100  ← 100 marta kichraydi!

Lekin daraxtlar to'liq mustaqil emas — ρ korrelyatsiya bor:
  Var(RF) = ρ·σ² + (1-ρ)·σ²/N

  ρ kichik (Feature randomness tufayli) →
  (1-ρ)·σ²/N kichik →
  Umumiy variance kichik → Barqarorlik yuqori!
```

---

6. Out-of-Bag (OOB) Baholash

```
Bootstrap da har daraxt ~36.8% nuqtani ko'rmaydi
Bu nuqtalar → OOB namunasi → Bepul validatsiya!

Har nuqta uchun:
  Uni ko'rmagan daraxtlar → Ovoz beradi
  Bu ovozlarning to'g'riligi → OOB skori

OOB skori ≈ Cross-validation skori
Lekin qo'shimcha hisoblash kerak emas!

Amaliy foyda:
  → Train/Test ajratish shart emas (kichik ma'lumotda foydali)
  → n_estimators (daraxtlar soni) ni sozlashda ishlatiladi
```

```python
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,
    oob_score=True,     # OOB baholashni yoqish
    random_state=42
)
model.fit(X_train, y_train)
print(f"OOB Score: {model.oob_score_:.4f}")
```

---

7. Feature Importance — Nazariy Asos

Feature Importance — har o'zgaruvchining bashorat sifatiga qo'shgan hissasini o'lchaydigan ko'rsatkich.

Mean Decrease in Impurity (MDI):

```
G'oya:
  Har bo'lishda Gini yoki Entropy kamayishi hisoblanadi
  Qaysi o'zgaruvchi bo'lishda ishlatilgan? → Unga hissa beriladi
```

```
    Barcha daraxtlar bo'yicha o'rtacha → Feature Importance

  Formula:
    FI(xⱼ) = Σ   Σ  [nₜ/n · ΔImpurity(t, xⱼ)]
             trees t∈T(xⱼ)

    nₜ → t tugunidagi nuqtalar soni
    n  → jami nuqtalar soni
    ΔImpurity → bo'lish tufayli kamaygan impurity

  Normalizatsiya:
    Barcha FI lar yig'indisi = 1.0
    FI(xⱼ) ∈ [0, 1]

  Talqin:
    FI = 0.45 → Bu o'zgaruvchi modelning 45% ni tushuntiradi
    FI = 0.01 → Bu o'zgaruvchi deyarli muhim emas
```

Permutation Importance — Alternativ:

```
  G'oya: "Agar bu o'zgaruvchini aralashtirsa, model qanchalik yomonlashadi?"

  1. Asl model skori: 0.92
  2. X₃ o'zgaruvchini tasodifiy aralashtir
  3. Yangi model skori: 0.71
  4. Farq: 0.92 - 0.71 = 0.21 → X₃ muhim!

  5. X₇ o'zgaruvchini tasodifiy aralashtir
  6. Yangi model skori: 0.91
  7. Farq: 0.92 - 0.91 = 0.01 → X₇ muhim emas!

  MDI vs Permutation:
    MDI → Tez, lekin yuqori kardinallik (ko'p qiymat)
          o'zgaruvchilarni haddan oshiq baholaydi
    Permutation → Sekin, lekin ishonchliroq
```

---

8. Feature Importance Qanday Ishlatiladi?

1. Keraksiz o'zgaruvchilarni olib tashlash:

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Feature importance olish
importances = pd.Series(
    model.feature_importances_,
    index=X.columns
).sort_values(ascending=False)

# Vizualizatsiya
```

```python
importances.plot(kind="bar", figsize=(10, 5), color="steelblue")
plt.title("Feature Importance")
plt.ylabel("Muhimlik darajasi")
plt.tight_layout()
plt.show()

# Muhim emas o'zgaruvchilarni olib tashlash
muhim = importances[importances > 0.05].index
X_yangi = X[muhim]
```

2. Domenni tushunish:

```
Tibbiyot modelida:
  Feature Importance:
    Yoshlik: 0.35 → Eng muhim
    Arterial bosim: 0.28
    Qon shakari: 0.21
    Jinsi: 0.10
    Vazni: 0.06

  Xulosa: "Kasallik xavfida yosh va qon bosimi eng muhim
           omillar ekan" → Tibbiy tadqiqot yo'nalishi!
```

3. Multikolinearlik aniqlash:

```
Agar 2 ta o'zgaruvchi kuchli korrelyatsiyada bo'lsa:
  Ikkalasining FI past ko'rinadi (bir-birining ulushini oladi)
  → Bittasini olib tashlash → Boshqasining FI oshadi
  → Model xira ko'rsatmalar beradi
```

---

9. Random Forest Giperparametrlari

```
n_estimators (Daraxtlar soni):
  Ko'proq → Yaxshiroq, lekin sekinroq
  100-500 → Ko'p holat uchun yetarli
  OOB skori plateau ga chiqqanda → Yetarli

max_features (Har bo'lishda o'zgaruvchilar):
  "sqrt" → √p (klassifikatsiya standarti)
  "log2" → log₂(p)
  Kichik → Ko'proq tasodifiylik → Past korrelyatsiya
  Katta  → Kam tasodifiylik → Decision Tree ga yaqinlashadi

max_depth:
  None → Barglar toza bo'lguncha (RF da odatda OK)
  RF bagging tufayli overfitting ga chidamliroq

min_samples_leaf:
  Katta → Sodda daraxtlar → Sekinroq overfit
  Kichik ma'lumotda oshirish foydali

n_jobs=-1:
  Barcha CPU yadrolari → Parallel hisoblash → Tez
```

10. RF vs Decision Tree — To'liq Taqqoslash

Xususiyat	Decision Tree	Random Forest
Beqarorlik	❌ Yuqori	✅ Past
Overfitting	❌ Moyil	✅ Chidamli
Aniqlik	⚠️ O'rtacha	✅ Yuqori
Tushunarliligi	✅ Grafik chiziladi	❌ "Qora quti"
Tezlik	✅ Tez	⚠️ Sekinroq
Feature Importance	✅ Bor	✅ Ishonchliroq
Kichik ma'lumot	⚠️	✅ OOB tufayli
Ko'p o'zgaruvchi	⚠️	✅ Feature random.

---

11. Xulosa

```
Random Forest = Bagging + Feature Randomness

Bagging:
  → Har daraxt tasodifiy bootstrap namunada o'rganadi
  → Klassifikatsiya: ovoz berish
  → Regressiya: o'rtacha

Feature Randomness:
  → Har bo'lishda faqat m=√p o'zgaruvchi ko'riladi
  → Daraxtlar o'zaro mustaqil → Korrelyatsiya past
  → O'rtalashtirish samarali

Decision Tree dan ustunligi:
  ✅ Variance kamayadi (σ²/N)
  ✅ Overfitting ga chidamli
  ✅ OOB — bepul validatsiya
  ✅ Feature Importance — ishonchli

Feature Importance:
  → Har o'zgaruvchining Gini kamaytirish hissasi
  → Keraksiz o'zgaruvchilarni aniqlash
  → Domen bilimlarini tasdiqlash
  → Normalizatsiya: Σ FI = 1.0
```

> 💡 **Eslab qol:** Random Forest — "yomon o'quvchilar jamoasi emas, **har biri turli narsani yaxshi biladigan mutaxassislar jamoasi**". Har daraxt o'zining bootstrap va feature to'plamida mutaxassis — birlashganda esa hech biri yolg'iz erisholmaydigan aniqlikka erishiladi!

🔴 **SAVOL 28:**

KLASSIFIKATSIYA MODELLARINI BAHOLASH METRIKALARI: `ACCURACY`, `PRECISION`, `RECALL`, `F1-SCORE` VA `CONFUSION MATRIX`. BEMORLARNI TASHXISLASHDA `PRECISION` VA `RECALL` DAN QAYSI BIRI MUHIMROQ?

---

1. Confusion Matrix — Hamma Metrikalarning Asosi

Barcha klassifikatsiya metrikalari **Confusion Matrix** dan kelib chiqadi. Avval uni mukammal tushunish kerak.

```
Ikkilik klassifikatsiyada to'rtta holat:

                BASHORAT
                Musbat (1)   Manfiy (0)
             ┌──────────────┬──────────────┐
Musbat (1)   │      TP      │      FN      │
HAQIQIY      ├──────────────┼──────────────┤
Manfiy (0)   │      FP      │      TN      │
             └──────────────┴──────────────┘

TP (True Positive)  → Haqiqiy 1, bashorat 1 ✅ To'g'ri musbat
TN (True Negative)  → Haqiqiy 0, bashorat 0 ✅ To'g'ri manfiy
FP (False Positive) → Haqiqiy 0, bashorat 1 ❌ Noto'g'ri musbat (I tur xato)
FN (False Negative) → Haqiqiy 1, bashorat 0 ❌ Noto'g'ri manfiy (II tur
xato)
```

FP va FN — qaysi biri yomon?

```
Bu KONTEKSTGA bog'liq — eng muhim nazariy tushuncha!

FP (I tur xato) — "Yolg'on signalizatsiya"
  Sog' odamni kasal deyish
  Spam bo'lmagan xatni spam deyish
  Zararsiz faylni virus deyish

FN (II tur xato) — "O'tkazib yuborish"
  Kasal odamni sog' deyish ← Tibbiyotda HALOKATLI!
  Spam xatni inbox ga o'tkazish
  Virusni o'tkazib yuborish

Kontekst qaysi xato yomonroq ekanini belgilaydi!
```

---

2. Accuracy — Umumiy To'g'rilik

```
Formula:
  Accuracy = (TP + TN) / (TP + TN + FP + FN)

Talqin:
  "Barcha bashoratlarning qanchasi to'g'ri?"

Misol: 100 ta bemor
```

```
  TP=45, TN=50, FP=3, FN=2
  Accuracy = (45+50)/(45+50+3+2) = 95/100 = 95%
```

Accuracy ning katta muammosi — Imbalanced Classes:

```
Muammo: Nomutanosib ma'lumot

Aytaylik: 100 ta bemor
  95 ta sog' (sinf 0)
  5 ta kasal (sinf 1)

Ahmoq model: "Hamma sog'" deydi
  TP=0, TN=95, FP=0, FN=5
  Accuracy = (0+95)/100 = 95% ← Ajoyib ko'rinadi!

Lekin model birorta kasalni topmadi!
  → Accuracy bu holatda mutlaqo ma'nosiz

Xulosa:
  Balanced ma'lumot   → Accuracy foydali
  Imbalanced ma'lumot → Accuracy ALDAMCHI
  Tibbiyot, firibgarlik, anomaliya → HECH QACHON faqat Accuracy
```

---

3. Precision — Aniqlik

```
Formula:
  Precision = TP / (TP + FP)

Talqin:
  "Model 'kasal' degan bemorlarning qanchasi haqiqatan kasal?"
  "Musbat degan bashoratlarimning qanchasi to'g'ri?"

Intuitsiya:
  → "Ishonch darajasi" — model musbat deganda qanchalik ishonsa bo'ladi?
  → FP ni kamaytirish muhim bo'lganda Precision muhim

Misol:
  Model 50 ta odamni "kasal" dedi
  Ulardan 40 tasi haqiqatan kasal
  Precision = 40/50 = 0.80 = 80%

  "Model kasal degan odamlarning 80% i haqiqatan kasal"
```

Precision qachon muhim:

```
Spam filtri:
  FP = Muhim xat spam deb belgilandi → Foydalanuvchi yo'qotadi
  → FP kamaysin → Precision yuqori bo'lsin

Qidiruv tizimi:
  FP = Keraksiz natijalar chiqdi → Foydalanuvchi vaqti ketadi
  → Precision muhim
```

```
Jinoiy sudda:
  FP = Begunoh odam qamaladi ← Adolatsizlik!
  → "Shubhali bo'lsa ham 100% isbotlanmasa qo'y"
  → Precision juda yuqori bo'lsin
```

---

4. Recall — Qamrov (Sezgirlik)

```
Formula:
  Recall = TP / (TP + FN)

Talqin:
  "Haqiqatan kasal bemorlarning qanchasi aniqlanadi?"
  "Barcha musbatlarni qanchasini topdim?"

Sinonimlar:
  Sensitivity (Sezgirlik) — tibbiyotda
  True Positive Rate (TPR)
  Hit Rate

Misol:
  100 ta haqiqiy kasal bemor bor
  Model ulardan 85 tasini "kasal" dedi
  Recall = 85/100 = 0.85 = 85%

  "Barcha kasallarning 85% i aniqlandi"
  15% o'tkazib yuborildi (FN)
```

Recall qachon muhim:

```
Saraton skriningi:
  FN = Kasal odam "sog'" deb yuborildi ← HALOKATLI!
  → FN kamaysin → Recall yuqori bo'lsin

Zilzila bashorati:
  FN = Zilzila bashorat qilinmadi ← Odamlar vafot etadi
  → Recall muhim

Terrorchilarni aniqlash:
  FN = Terrorchi o'tkazib yuborildi ← Xavfli
  → Recall muhim

Virusni aniqlash (antivirus):
  FN = Virus o'tkazib yuborildi ← Zarar
  → Recall muhim
```

---

5. Precision va Recall — Tradeoff

Bu ikki metrika **o'zaro ziddiyatli** — birini oshirish ikkinchisini tushiradi:

```
Chegara (Threshold) = 0.5 (standart):
  Precision = 0.80,  Recall = 0.75
```

```
Chegarani oshirish → 0.7:
  Model faqat juda ishonchli holatlarda "musbat" deydi
  TP kamayadi, FP kamayadi, FN oshadi
  Precision ↑ = 0.90,  Recall ↓ = 0.55

Chegarani kamaytirish → 0.3:
  Model deyarli hamma narsani "musbat" deydi
  TP oshadi, FP oshadi, FN kamayadi
  Precision ↓ = 0.60,  Recall ↑ = 0.95

Vizual:
  Precision
  1.0 ──────────\
                ──────\
  0.5                 ──────\
                            ──────── Recall
       0.0    0.5    1.0
```

Hayotiy analogiya:

```
Baliqchi to'ri:

Katta ko'z to'r (Threshold yuqori):
  → Ko'p baliq o'tib ketadi (Recall ↓ = FN ko'p)
  → Lekin to'rdagi hamma narsa baliq (Precision ↑ = FP kam)

Kichik ko'z to'r (Threshold past):
  → Deyarli hech narsa o'tib ketmaydi (Recall ↑ = FN kam)
  → Lekin to'rda baliqdan tashqari axlat ham bor (Precision ↓ = FP ko'p)
```

---

6. F1-Score — Muvozanatli Metrika

```
Formula:
  F1 = 2 · (Precision · Recall) / (Precision + Recall)

Bu — Harmonik o'rtacha (Arithmetic o'rtachadan farqli)

Nima uchun Harmonik o'rtacha?
  Arithmetic: (0.9 + 0.1) / 2 = 0.5
  Harmonik:   2·(0.9·0.1)/(0.9+0.1) = 0.18

  Harmonik o'rtacha kichik qiymatga og'adi →
  Agar Precision yoki Recall dan biri past bo'lsa →
  F1 ham past bo'ladi → Bu adolatli!

Misol:
  Model A: Precision=0.90, Recall=0.90 → F1 = 0.90
  Model B: Precision=0.99, Recall=0.50 → F1 = 0.66
  Model C: Precision=0.50, Recall=0.99 → F1 = 0.66

  B va C "aldamchi" — bir metrika yuqori, boshqasi past
  F1 bu aldovni fosh qiladi!
```

```
Qachon F1 ishlatish:
  → Imbalanced ma'lumot
  → Precision va Recall ikkalasi muhim
  → Bitta raqam bilan taqqoslash kerak
```

F-beta Score — Umumlashtirilgan holat:

```
F_β = (1+β²) · (Precision·Recall) / (β²·Precision + Recall)

β < 1 → Precision muhimroq (β=0.5: Precision 2x muhim)
β = 1 → Teng (oddiy F1)
β > 1 → Recall muhimroq (β=2: Recall 2x muhim)

Tibbiyotda: F₂ score → Recall 2x muhim
Sudda:      F₀.₅ score → Precision 2x muhim
```

---

7. ROC Curve va AUC

```
ROC (Receiver Operating Characteristic) Curve:
  X o'qi: FPR = FP/(FP+TN)  ← "Yolg'on signal" darajasi
  Y o'qi: TPR = Recall       ← "Haqiqiy signal" darajasi

  Threshold 0→1 bo'yicha har qiymat uchun (FPR, TPR) nuqta →
  Barcha nuqtalarni ulash → ROC Curve

AUC (Area Under Curve):
  ROC Curve ostidagi maydon
  AUC = 1.0 → Mukammal model
  AUC = 0.5 → Tasodifiy model (diagonal chiziq)
  AUC = 0.0 → Teskari model (amalda mumkin emas)

Talqin:
  AUC = 0.85 → "Tasodifiy tanlangan musbat nuqta,
                tasodifiy tanlangan manfiy nuqtadan
                85% ehtimollik bilan yuqori skor oladi"

AUC qachon foydali:
  → Threshold tanlamay umumiy model sifatini o'lchash
  → Turli modellarni solishtirish
  → Imbalanced ma'lumotda Accuracy o'rniga
```

---

8. Ko'p Sinf Holati

```
Ko'p sinfda (3+) Confusion Matrix:
```

	Bashorat: Kasal	Bashorat: Sog'	Bashorat: Shubhali
HAQIQIY: Kasal	45	3	2
HAQIQIY: Sog'	2	48	0

```
Shubhali  │   3   │  1   │   46    │
          └───────┴──────┴─────────┘

Har sinf uchun alohida Precision, Recall, F1 hisoblanadi.

Agregatsiya usullari:
  Macro average:    Har sinf teng vaznda — minoritar sinf muhim
  Weighted average: Har sinf miqdoriga qarab vazn — ko'p sinfga e'tibor
  Micro average:    Barcha TP, FP, FN yig'iladi — imbalanced uchun
```

---

9. Bemorlarni Tashxislashda Recall Muhimroq

Bu savolning to'g'ri javobi va **chuqur asoslash:**

```
Savol: Precision yuqori mi, Recall yuqori mi?
Javob: RECALL — quyidagi sabablar bilan:
```

Sabab 1 — FN ning oqibati FP dan og'irroq:

```
FP (Sog' odamni kasal deyish):
  → Qo'shimcha tekshiruv buyuriladi
  → Bemor biroz tashvishlanadi
  → Material zarar: qo'shimcha tahlil xarajati
  → Lekin: To'g'ri tashxis qo'yilgach, muammo hal bo'ladi

FN (Kasal odamni sog' deyish):
  → Kasallik aniqlanmaydi → Davolanmaydi
  → Kasallik rivojlanib ketadi
  → Kech aniqlangan saraton → Davolash qiyin
  → Yurak kasalligi o'tkazib yuborildi → Infarkt
  → Natija: BEMOR HAYOTINI XATaRGA QO'YISH

Xulosa:
  FP → Pul va vaqt sarfi (tuzatuvchan)
  FN → Hayot xavfi (qaytmas oqibat)
```

Sabab 2 — Tibbiy etika:

```
"Primum non nocere" — Lotincha tibbiy qoida:
"Avval zarar keltirma"

Kasalni o'tkazib yuborish → Zarar keltirishdan YOMONI
Sog' odamni tekshiruv uchun yuborish → Kichik noqulaylik

Tibbiy amaliyotda:
  Sezgirlik (Recall) = Sensitivity → Birinchi o'rinda
  Aniqlik (Precision) = Specificity → Ikkinchi o'rinda
```

Sabab 3 — Skrining va Diagnostika farqi:

```
Skrining (Birinchi bosqich):
  Maqsad: Hech kimni o'tkazib yuborma
```

```
      → Recall MAKSIMAL bo'lsin (hatto 99%+)
      → FP oshsa ham mayli → Keyingi tekshiruv aniqlaydi
      Misol: Mammografiya, PSA testi, COVID skreening

    Diagnostika (Ikkinchi bosqich):
      Maqsad: Aniq tashxis qo'y
      → Precision ham muhim
      → Biopsy, MRI, qo'shimcha tahlillar
      Misol: Saraton biopsiyasi

    Ikki bosqichli tizim:
      Skrining (Recall ↑) → Shubhalilar → Diagnostika (Precision ↑)
      Bu tibbiyotdagi optimal yondashuv!
```

Sabab 4 — Asimmetrik zarar:

```
    Zarar funksiyasi:
      C(FP) = 1  ← Noto'g'ri tekshiruv yuborish → 1 birlik zarar
      C(FN) = 50 ← Kasalni o'tkazib yuborish → 50 birlik zarar!

    Bunday asimmetriyada:
      Umumiy zarar = C(FP)·FP + C(FN)·FN
      → FN ning koeffitsienti katta → FN minimallash kerak
      → FN minimallash = Recall maksimallashtirish!

    F_β score bilan:
      β=2 → Recall 2x muhim deb belgilash
      β=5 → Recall 5x muhim deb belgilash
```

---

10. Amaliy Misol

```python
from sklearn.metrics import (
    accuracy_score, precision_score,
    recall_score, f1_score,
    confusion_matrix, classification_report
)
import numpy as np

y_haqiqiy  = [1,1,0,1,0,0,1,1,0,1]
y_bashorat = [1,1,0,0,0,0,1,1,1,1]

# Asosiy metrikalar
print(f"Accuracy:  {accuracy_score(y_haqiqiy, y_bashorat):.3f}")
print(f"Precision: {precision_score(y_haqiqiy, y_bashorat):.3f}")
print(f"Recall:    {recall_score(y_haqiqiy, y_bashorat):.3f}")
print(f"F1:        {f1_score(y_haqiqiy, y_bashorat):.3f}")

# To'liq hisobot
print(classification_report(y_haqiqiy, y_bashorat,
      target_names=["Sog'", "Kasal"]))

# Tibbiyot uchun: Recall ni oshirish
# Threshold ni kamaytirish:
ehtimolliklar = model.predict_proba(X_test)[:, 1]
```

```python
threshold = 0.3   # 0.5 o'rniga → Recall oshadi
y_pred_tibbiy = (ehtimolliklar >= threshold).astype(int)
```

---

11. Xulosa

```
Confusion Matrix:
  TP, TN, FP, FN → Barcha metrikalarning asosi
  FP = I tur xato (Yolg'on musbat)
  FN = II tur xato (O'tkazib yuborish)

Metrikalar:
  Accuracy  = (TP+TN)/n        → Balanced ma'lumotda
  Precision = TP/(TP+FP)       → FP muhim bo'lganda
  Recall    = TP/(TP+FN)       → FN muhim bo'lganda
  F1        = 2·P·R/(P+R)      → Ikkalasi muhim, imbalanced

Tibbiyotda:
  RECALL muhimroq — 3 asosiy sabab:
  1. FN oqibati (hayot xavfi) >> FP oqibati (qo'shimcha tekshiruv)
  2. Tibbiy etika: "Kasalni o'tkazib yubOrma"
  3. Skrining: Recall↑, Diagnostika: Precision↑

Threshold boshqaruvi:
  Threshold ↓ → Recall ↑, Precision ↓
  Threshold ↑ → Precision ↑, Recall ↓
  β parametri → Qaysi biri muhimroqligini belgilash
```

> 💡 **Eslab qol:** Metrika tanlash — **texnik emas, etik qaror**. "Qaysi xato yomonroq?" savoliga javob → Biznes yoki hayot konteksti beradi, matematik formula emas. Tibbiyotda bu savol: "Kasalni o'tkazib yuborish yomonmi, sog'ni davolanishga yuborish yomonmi?" → Javob aniq: Birinchisi!

🔴 **SAVOL 29:**

POLINOM REGRESSIYASI VA CHIZIQLI REGRESSIYA O'RTASIDAGI FARQ. YUQORI DARAJALI POLINOMLAR QANDAY MUAMMOLARGA OLI KELADI? `FEATURE ENGINEERING` NIMA?

---

1. Chiziqli Regressiyaning Cheklovi

Chiziqli regressiya faqat **to'g'ri chiziq** munosabatini modellashtiira oladi:

```
Y = β₀ + β₁X

Grafik:
Y │        /
  │      /
  │    /        ← To'g'ri chiziq
  │  /
  │/
```

```
   └──────────── X

Hayotiy munosabatlar ko'pincha CHIZIQLI EMAS:
  Dori dozasi vs ta'sir     → S-shakl
  Iqtisodiy o'sish          → Egri chiziq
  Fizik harakat             → Qavariq/botiq egri
  Narx vs talab             → Giperbol

Chiziqli model bu munosabatlarni ushlab OLOLMAYDI!
```

---

2. Polinom Regressiyasi — G'oya

Polinom regressiyasi — X ning darajalarini yangi o'zgaruvchi sifatida qo'shib, egri munosabatlarni modellash.

```
Darajasi 1 (Chiziqli):
  Y = β₀ + β₁X

Darajasi 2 (Kvadratik):
  Y = β₀ + β₁X + β₂X²

Darajasi 3 (Kubik):
  Y = β₀ + β₁X + β₂X² + β₃X³

Darajasi n:
  Y = β₀ + β₁X + β₂X² + ... + βₙXⁿ
```

Muhim nazariy tushuncha:

```
Polinom regressiyasi — aslida CHIZIQLI regressiya!

Y = β₀ + β₁X₁ + β₂X₂ + β₃X₃
bu yerda: X₁=X,  X₂=X²,  X₃=X³

Chiziqli regressiyadan farqi — faqat X lar yaratilish usulida.
O'rgatish algoritmi bir xil (OLS).
"Polinom" — chiziqli emas, degani emas!
```

Grafik ko'rinish:

```
Chiziqli:          Kvadratik:         Kubik:
Y │    /           Y │    /\         Y │   /\
  │  /               │  /  \           │  /  \
  │/                 │ /    \          │ /    \
  └──── X            │/      \         └──────── X
                     └──────── X          (S-shakl)
```

---

3. Polinom Darajasi — Naqsh va Overfitting

Bu savolning **eng muhim qismi:**

```
Degree=1 (Chiziqli):   Juda sodda → Underfitting
Degree=2 (Kvadratik):  Ko'pchilik egri munosabatlar → Optimal
Degree=3 (Kubik):      Murakkab shakllar → Qabul qilinadi
Degree=5+:             Haddan oshiq → Overfitting xavfi kuchli
Degree=n-1:            n ta nuqtadan o'tadigan chiziq → 100% overfit!

n ta nuqtani ulaydigan polinom:
Y │ *     *       Har nuqtadan o'tadi
  │  \   / \      → Train MSE = 0.0
  │   \ /   \     → Test MSE = cheksiz katta!
  │    *     *
  └──────────── X
```

Nazariy asos — Runge fenomeni:

```
Yuqori darajali polinom chegaralarda keskin "otib chiqadi"
(oscillation – tebranish).

Daraja 10 polinom:
  O'rta nuqtalarda → yaxshi
  Chekkalar yaqinida → portlovchi tebranish

Bu matematik haqiqat: Daraja oshgan sari
intervalning chekkalarida polinom beqarorroq bo'ladi.
```

---

4. Yuqori Darajali Polinomlarning Muammolari

Muammo 1 — Overfitting:

```
Kuzatuvlar: 10 ta nuqta
Degree=9 polinom: 10 ta parametr → 10 ta nuqtadan aniq o'tadi!

Train R²= 1.00  ← Mukammal (yodlagan)
Test  R²= -3.45 ← Falokatli (yangi nuqtalarda portlaydi)

Polinom "shovqinni ham qoida sifatida" yodlaydi.
```

Muammo 2 — Multikolinearlik:

```
X = [1, 2, 3, 4, 5]
X² = [1, 4, 9, 16, 25]
X³ = [1, 8, 27, 64, 125]

X, X², X³ bir-biri bilan juda kuchli korrelyatsiyali!
→ OLS matritsasi (XᵀX) deyarli singular (teskari qilib bo'lmaydi)
→ Koeffitsiyentlar beqaror, ulkan, musbat/manfiy almashadi
→ Talqin qilish MUMKIN EMAS

Yechim: Polinom xususiyatlarni standartlashtirish
        yoki Regularizatsiya (Ridge)
```

Muammo 3 — Ekstrapolyatsiya Xavfi:

```
Train ma'lumoti: X ∈ [0, 10]
Model degree=8 polinom o'rganadi

X=11 (biroz tashqarida) → Bashorat portlaydi:
  Chiziqli: Y≈ to'g'ri chiziq davomi (mantiqli)
  Degree=8: Y≈ ±∞ (beqaror tebranish!)

Polinom regressiyasi INTERPOLATSIYA (ichki) da yaxshi,
EKSTRAPOLYATSIYA (tashqari) da XAVFLI!
```

Muammo 4 — Hisoblash murakkabligi:

```
Daraja oshganda:
  → Ko'proq parametrlar → Ko'proq xotira
  → Matritsa teskarlash qiyinlashadi
  → Raqamli beqarorlik (numerical instability)

Degree=20, n=100:
  20 ta polinom xususiyat → X matritsa 100×20
  XᵀX = 20×20 matritsa → Teskarlash beqaror bo'lishi mumkin
```

---

5. Optimal Daraja Tanlash

```
Usul 1 — Cross-Validation:
  Har degree uchun CV skori hisoblash
  CV skori maksimal bo'lgan degree → Optimal

  Degree:  1    2    3    4    5    6
  CV R²: 0.62 0.89 0.91 0.90 0.85 0.71
                     ↑
                 Optimal = 3

Usul 2 — Learning Curve:
  Yuqorida ko'rsatilganidek:
  Train va Validation egri chiziqlarining kesishishi → Optimal

Usul 3 — Regularizatsiya:
  Yuqori daraja + Ridge/Lasso regularizatsiya
  → Katta koeffitsiyentlarga jarima
  → Ortiqcha darajalar o'z-o'zidan "o'chadi"
```

---

6. Feature Engineering — Nazariy Asos

Feature Engineering — xom ma'lumotdan model uchun **yanada informativroq** xususiyatlar (features) yaratish san'ati.

```
Xom ma'lumot:
  Tug'ilgan yil: 1990
  Hozirgi yil:  2024

Feature Engineering:
```

```
  Yosh = 2024 - 1990 = 34 ← Model uchun ancha foydali!

Nima uchun muhim:
  "Axlat kiritsa → Axlat chiqadi" (Garbage In, Garbage Out)
  Eng yaxshi algoritm ham yomon xususiyatlar bilan yomon natija beradi.
  O'rtacha algoritm + Yaxshi xususiyatlar = Ajoyib natija!
```

Feature Engineering qayerda joylashadi:

```
Xom Ma'lumot
     ↓
Feature Engineering  ← Bu qism
     ↓
Model O'rgatish
     ↓
Baholash
```

---

7. Feature Engineering Turlari

Tur 1 — Matematik Transformatsiyalar:

```
# Polinom xususiyatlar:
X², X³, X₁·X₂ ← O'zaro ta'sir (interaction)

# Logarifmik (qiyralik taqsimot uchun):
log(Daromad) ← Daromad taqsimoti ko'pincha qiyiq

# Kvadrat ildiz:
√(Masofa) ← Katta qiymatlar ta'sirini kamaytirish

# Teskari:
1/X ← Teskari proporsional munosabat

# Eksponensial:
e^X ← Tez o'suvchi jarayonlar
```

Nima uchun log transformatsiya:

```
Asl daromad:
  [1000, 2000, 5000, 100000, 200000]
  Tarqoqlik katta → Model katta qiymatlarga o'rganadi

log(daromad):
  [6.9, 7.6, 8.5, 11.5, 12.2]
  Tarqoqlik kichik → Model barcha qiymatlarni teng ko'radi

Qachon log:
  → Qiyralik taqsimot (right-skewed)
  → Narxlar, daromad, aholi
  → Protsent o'zgarishlar muhim bo'lsa
```

Tur 2 — Vaqt Seriyasi Xususiyatlari:

```
# Xom: "2024-03-15 14:30:00"

# Yaratilgan xususiyatlar:
Yil    = 2024
Oy     = 3          ← Mavsumiylik
Hafta_kuni = 4      ← Ish kuni vs dam olish
Soat   = 14         ← Qism-kunlik naqsh
Kvartal = 1         ← Biznes davri
Hafta_soni = 11     ← Yillik naqsh

# Tsiklik xususiyatlar (oy 12 dan keyin 1 ga qaytadi):
sin(2π·Oy/12)    ← Tsiklik xarakter
cos(2π·Oy/12)    ← Model "Dekabr va Yanvar yaqin" ni tushunadi
```

Nima uchun sin/cos:

```
Oy: 1, 2, 3, ..., 11, 12, 1, 2, ...
    (Yanvar va Dekabr qo'shnilar!)

Oddiy kodlash: 1 va 12 → |12-1|=11 farq (lekin aslida 1 kun!)
sin/cos kodlash: Yanvar va Dekabr → Yaqin qiymatlar ✅
```

Tur 3 — Kategoriyali O'zgaruvchilar:

```
# One-Hot Encoding:
Shahar: ["Toshkent", "Samarqand", "Buxoro"]
→
  Toshkent: [1, 0, 0]
  Samarqand:[0, 1, 0]
  Buxoro:   [0, 0, 1]

# Label Encoding (Tartibli):
Ta'lim: ["Maktab", "Bakalavr", "Magistr", "Doktor"]
→      [    0,         1,          2,        3    ]
(Tartib muhim bo'lganda)

# Target Encoding:
Har kategoriyani o'sha kategoriya uchun Y o'rtachasi bilan almashtirish
(Katta kardinallik uchun foydali)
```

Tur 4 — O'zaro Ta'sir Xususiyatlari:

```
# Ikki o'zgaruvchining mahsuloti:
Maydon × Qavatlar = Umumiy_maydon
Narx × Miqdor     = Jami_summa
Yosh × Tajriba    = Tajriba_yosh_nisbati

# Nisbatlar:
BMI = Vazn / Boy²
Daromad_soliq_nisbati = Soliq / Daromad

# Farqlar:
Hozirgi_narx - Avvalgi_narx = O'zgarish
Max_qiymat - Min_qiymat     = Amplituda
```

Tur 5 — Agregatsiya Xususiyatlari:

```
# Mijoz tarixidan:
Oxirgi_30_kun_xarid_soni     ← Faollik
O'rtacha_chek_summasi        ← Xarid kuchi
Eng_ko'p_xarid_kategoriya    ← Qiziqishlar
Oxirgi_xariddan_kunlar       ← Sovuqlik (Recency)

# Oyna funksiyalari (Time Series):
Oxirgi_7_kun_o'rtacha        ← Qisqa trend
Oxirgi_30_kun_o'rtacha       ← Uzun trend
Скользящее_standart_og'ish   ← Beqarorlik
```

---

8. Feature Engineering vs Feature Selection

```
Feature Engineering:
  → YANGI xususiyatlar YARATISH
  → Domain bilimi kerak
  → Kreativ jarayon
  → Xom ma'lumot → Informativ xususiyat

Feature Selection:
  → Mavjud xususiyatlardan TANLASH
  → Keraksizlarni OLIB TASHLASH
  → Avtomatik (Random Forest FI, Lasso) yoki qo'lda
  → Ko'p xususiyat → Kerakli xususiyatlar

Ikkalasi birga:
  Feature Engineering → Ko'p potensial xususiyat
  Feature Selection   → Ulardan eng yaxshilarini tanlash
```

---

9. Sklearn da Polinom Feature Engineering:

```python
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline
from sklearn.linear_model import Ridge
from sklearn.model_selection import cross_val_score

# Polinom xususiyatlar yaratish:
poly = PolynomialFeatures(
    degree=3,
    include_bias=False,    # β₀ ni LinearRegression o'zi qo'shadi
    interaction_only=False # X², X·Y ham qo'shsin
)

# X=[a, b] bo'lsa, degree=2 da:
# [a, b, a², ab, b²] → 5 ta xususiyat

# Pipeline — to'liq jarayon:
pipeline = Pipeline([
    ("poly",  PolynomialFeatures(degree=3)),
    ("model", Ridge(alpha=1.0))  # Regularizatsiya bilan!
])

# Cross-validation bilan daraja tanlash:
```

```python
for degree in range(1, 8):
    pipe = Pipeline([
        ("poly",  PolynomialFeatures(degree=degree)),
        ("model", Ridge(alpha=1.0))
    ])
    skorlar = cross_val_score(pipe, X, y, cv=5, scoring="r2")
    print(f"Degree={degree}: {skorlar.mean():.3f} ± {skorlar.std():.3f}")
```

---

10. Xulosa

```
Chiziqli vs Polinom Regressiya:
  Chiziqli  → To'g'ri chiziq, sodda, interpretable
  Polinom   → Egri, moslashuvchan, overfitting xavfi

Yuqori daraja muammolari:
  Overfitting      → Train yaxshi, Test yomon
  Multikolinearlik → Koeffitsiyentlar beqaror
  Ekstrapolyatsiya → Chekkada portlaydi
  Runge fenomeni   → Tebranish

Optimal daraja:
  Cross-validation → CV skori maksimal daraja
  Regularizatsiya  → Ridge + yuqori daraja ham ishlaydi

Feature Engineering:
  Xom ma'lumot → Informativ xususiyatlar yaratish
  Matematik transformatsiya (log, sqrt, poly)
  Vaqt xususiyatlari (oy, kun, tsiklik)
  Kategoriyali kodlash (one-hot, target)
  O'zaro ta'sir (mahsulot, nisbat)

Asosiy qoida:
  ✅ Domain bilimi + Ijodkorlik = Yaxshi Feature Engineering
  ✅ Daraja = 2-3 → Ko'pchilik amaliy holat uchun yetarli
  ✅ Polinom + Regularizatsiya → Overfittingdan himoya
```

> 💡 **Eslab qol:** Feature Engineering — mashinali o'rganishning **"sehrli qismi"**. Ko'p holatlarda yaxshi feature yaratish, murakkab algoritm tanlashdan ko'ra **ko'proq natija** beradi. Kaggle musobaqalarida g'oliblar sirri ko'pincha yangi algoritmda emas — **aqlli feature engineering** da!

🔴 **SAVOL 30:**

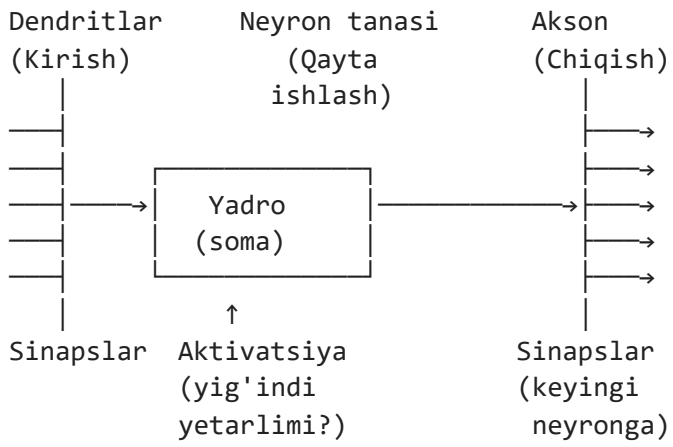
SUN'IY NEYRON TARMOQNING BIOLOGIK NEYRON BILAN O'XSHASHLIGI. `PERCEPTRON` MODELI, QATLAMLAR TUZILISHI VA AKTIVATSIYA FUNKSIYALARI (`RELU` , `SIGMOID` , `SOFTMAX`).

---

1. Biologik Neyron — Asos

Inson miyasida **86 milliard** neyron bor. Har neyron boshqalari bilan **10,000** gacha bog'lanish o'rnatadi. Sun'iy neyron tarmoq ana shu tuzilmani matematik jihatdan taqlid qiladi.

```
Biologik Neyron anatomiyasi:
```



Biologik jarayon:

```
1. Dendritlar → Qo'shni neyronlardan signal oladi
2. Sinaps → Signal kuchaytiriladi yoki zaiflashtiriladi (vaznlar!)
3. Neyron tanasi → Barcha signallarni YIG'ADI
4. Aktivatsiya → Yig'indi CHEGARADAN OSHSA → Signal yuboriladi
5. Akson → Signal keyingi neyronga o'tkaziladi
```

---

2. Biologik → Sun'iy Neyron: Parallel

Biologik	Sun'iy (Matematik)
Dendrit	↔ Kirish ($x_1, x_2, ..., x_n$)
Sinaps	↔ Vazn ($w_1, w_2, ..., w_n$)
Neyron tanasi	↔ Yig'indi funksiyasi ($\Sigma w_i x_i + b$)
Aktivatsiya	↔ Aktivatsiya funksiyasi f(z)
Akson	↔ Chiqish (output)
O'rganish	↔ Vaznlarni yangilash (Backpropagation)
Sinaps kuchi	↔ Vazn qiymati (w)
Inhibitatsiya	↔ Manfiy vazn
Qo'zg'alish	↔ Musbat vazn
Chegara	↔ Bias (b)

Muhim farqlar:

```
Biologik neyron:
  → Impuls (0 yoki 1) yuboradi
  → Millisekundlarda ishlaydi
  → Energiya samarali
  → O'rganish mexanizmi hali to'liq tushunilmagan

Sun'iy neyron:
  → Haqiqiy son (masalan 0.73) chiqaradi
  → Nanosekundlarda ishlaydi
  → Energiya ko'p sarflaydi
```

```
  → Backpropagation bilan o'rganadi (aniq matematika)

Xulosa: Sun'iy neyron — biologikni TAQLID qiladi,
        lekin aniq nusxa emas. Ilhom manba!
```

---

3. Perceptron — Eng Sodda Neyron

Perceptron — 1957 yilda Frank Rosenblatt tomonidan ixtiro qilingan. Eng sodda sun'iy neyron modeli.

```
Perceptron modeli:

x₁ ──w₁──┐
x₂ ──w₂──┤
x₃ ──w₃──┼──→ [Σwᵢxᵢ + b] ──→ f(z) ──→ Chiqish
...      │
xₙ ──wₙ──┘

Matematik:
  z = w₁x₁ + w₂x₂ + ... + wₙxₙ + b
  z = Wᵀx + b       ← Matritsa ko'rinishi
  ŷ = f(z)          ← Aktivatsiya funksiyasi
```

Vaznlar va Bias:

```
Vaznlar (w):
  → Har kirishning muhimligini belgilaydi
  → w katta (musbat) → Bu kirish muhim, oshiradi
  → w katta (manfiy) → Bu kirish muhim, kamaytiradi
  → w ≈ 0            → Bu kirish muhim emas

Bias (b):
  → Neyronning "qo'shimcha erkinligi"
  → Barcha kirishlar 0 bo'lganda ham chiqish 0 bo'lmasligi
  → Qaror chegarasini siljitish imkoni
  → b yo'q bo'lsa → Qaror chizig'i doim origindan o'tadi
```

Perceptron cheklovi — XOR muammosi:

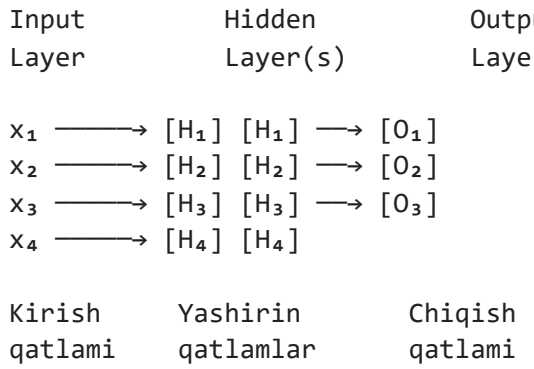
```
AND, OR → Perceptron hal qiladi (chiziqli ajratiladi)
XOR     → Perceptron HAL QILA OLMAYDI! (chiziqli ajratilmaydi)

XOR:
  (0,0)→0,  (0,1)→1,  (1,0)→1,  (1,1)→0

  Grafik:
  1 │  ●       ○
    │
  0 │  ○       ●
    └──────────────
       0        1
  Bitta to'g'ri chiziq bu 4 nuqtani ajrata olmaydi!

Yechim → Ko'p qatlamli tarmoq (MLP)
```

4. Ko'p Qatlamli Perceptron (MLP) — Qatlamlar Tuzilishi



Kirish qatlami (Input Layer):

```
→ Ma'lumotni tarmoqqa kiritadi
→ Hech qanday hisoblash bajarMaydi
→ Neyronlar soni = O'zgaruvchilar soni (features)
→ Masalan: 28×28 px rasm → 784 kirish neyron
```

Yashirin qatlamlar (Hidden Layers):

```
→ Asosiy hisoblash bu yerda!
→ Xususiyatlarni ierarxik tarzda o'rganadi:

1-qatlam: Oddiy naqshlar (chiziqlar, burchaklar)
2-qatlam: Murakkabrok naqshlar (shakl qismlari)
3-qatlam: Yuqori darajali tushunchalar (butun shakl)

"Deep" learning → Ko'p yashirin qatlam demak
```

Chiqish qatlami (Output Layer):

```
Klassifikatsiya (binary):
  → 1 neyron + Sigmoid → [0, 1] ehtimollik

Klassifikatsiya (ko'p sinf):
  → K neyron + Softmax → K ta ehtimollik (yig'indisi=1)

Regressiya:
  → 1 neyron + Aktivatsiyasiz (yoki Linear) → Haqiqiy son
```

To'liq bog'langan qatlam (Fully Connected):

```
Har neyron → Keyingi qatlamdagi BARCHA neyronlarga ulangan

Parametrlar soni:
  Qatlam 1: 4 neyron
  Qatlam 2: 3 neyron
  Bog'lanishlar: 4×3 = 12 ta vazn + 3 ta bias = 15 parametr

Katta tarmoq:
```

```
  784 → 256 → 128 → 10
  784×256 + 256×128 + 128×10 = 233,984 parametr!
```

---

5. Nima Uchun Aktivatsiya Funksiyasi Kerak?

```
Aktivatsiyasiz tarmoq:

  Qatlam 1: z₁ = W₁x + b₁
  Qatlam 2: z₂ = W₂z₁ + b₂ = W₂(W₁x + b₁) + b₂
                           = (W₂W₁)x + (W₂b₁ + b₂)
                           = W'x + b'  ← YaNa chiziqli!

Xulosa: Aktivatsiyasiz 100 qatlam = 1 qatlam!
        Qanchalik ko'p qatlam bo'lmasin → Chiziqli model.

Aktivatsiya funksiyasi → NOCHIZIQLILIK kiritadi →
Tarmoq murakkab naqshlarni o'rgana oladi!
```

---

6. ReLU — Eng Ko'p Ishlatiladigan

ReLU (Rectified Linear Unit) — zamonaviy chuqur tarmoqlarning standarti.

```
Formula:
  ReLU(z) = max(0, z)

Grafik:
   │        /
   │       /
   │      /
   │     /
───┼────/────────
   │    0
   │ (manfiy z → 0)

Xususiyatlar:
  z > 0 → z ni o'tkazadi (o'zgarmaydi)
  z ≤ 0 → 0 qaytaradi ("o'chiradi")
```

Nima uchun ReLU zo'r:

```
1. Hisoblash juda oson:
   max(0, z) → Bitta solishtirish
   Sigmoid/tanh dan 6x tezroq!

2. Gradient yo'qolishi muammosi YO'Q:
   z > 0 da gradient = 1 (yo'qolmaydi)
   Chuqur tarmoqlarda bu muhim!

3. Sparse aktivatsiya:
   ~50% neyronlar 0 chiqaradi → Tarmoq samarali
   Biologik neyronlarga o'xshash
```

```
4. Amalda ishlaydi:
   ImageNet, GPT, BERT → Barchasi ReLU ishlatadi
```

ReLU ning muammosi — Dying ReLU:

```
Muammo:
  Katta manfiy gradient → z juda kichik manfiy bo'lib qoladi
  → ReLU har doim 0 chiqaradi
  → Neyron "o'ladi" → O'rganmaydi!

Yechim — Leaky ReLU:
  LeakyReLU(z) = z,      z > 0 da
                 0.01z,  z ≤ 0 da
  (Manfiy qismda kichik qiyalik)

Yechim — ELU (Exponential Linear Unit):
  ELU(z) = z,          z > 0
           α(eᶻ - 1),  z ≤ 0
```

---

7. Sigmoid — Ehtimollik Uchun

```
Formula:
          1
σ(z) = ──────
       1 + e⁻ᶻ

Grafik:
1 │         _________
  │        /
0.5│      /
  │    /
0 │__/
  └──────────────────
     -∞    0    +∞
```

```
Chiqish: (0, 1) — Hech qachon 0 yoki 1 ga teng emas!
```

Sigmoid qachon ishlatiladi:

```
1. Binary klassifikatsiya chiqish qatlami:
   P(Y=1) = σ(z) → Ehtimollik talqini bor

2. LSTM va GRU darvozalari:
   "Qancha ma'lumot o'tkazish kerak?" → [0,1]

3. YASHIRIN qatlamlarda — ESKIRGAN:
   Gradient yo'qolishi muammosi:
   σ'(z) = σ(z)·(1-σ(z)) ≤ 0.25
   100 qatlam: 0.25^100 ≈ 10⁻⁶⁰ → Gradient yo'qoladi!
   → Shuning uchun yashirin qatlamlarda ReLU afzal
```

Sigmoid ning "Saturation" muammosi:

```
|z| katta bo'lsa:
  z = 10  → σ(10) ≈ 0.99999 → Gradient ≈ 0.00001
  z = -10 → σ(-10)≈ 0.00001 → Gradient ≈ 0.00001

  Gradient deyarli 0 → Neyron o'rganmaydi!
  Bu "Vanishing Gradient" muammosining asosiy sababi.
```

---

8. Softmax — Ko'p Sinf Klassifikatsiyasi

Softmax — K ta chiqishni K ta ehtimollikka aylantiradi (yig'indisi = 1).

```
Formula:
              e^z_j
P(j) = ──────────────
        Σ e^z_k
         k

Misol — Rasm klassifikatsiyasi (3 sinf):
  z = [2.0, 1.0, 0.1]   ← Xom chiqishlar

  e^2.0 = 7.389
  e^1.0 = 2.718
  e^0.1 = 1.105
  Jami = 11.212

  P("Mushuk") = 7.389/11.212 = 0.659 → 65.9%
  P("It")     = 2.718/11.212 = 0.242 → 24.2%
  P("Qush")   = 1.105/11.212 = 0.099 →  9.9%
  ──────────────────────────────────────────
  Jami                                = 100.0% ✅
```

Softmax xususiyatlari:

```
1. Yig'indi = 1 → To'g'ri ehtimollik taqsimoti
2. Eksponenta → Katta qiymat yanada kattaroq ulush oladi
   ("Winner takes more" — g'olib ko'proq oladi)
3. Barcha sinf o'zaro bog'liq:
   Bir sinfning ehtimolligi oshsa → Boshqalariniki kamayadi
4. Temperatur parametri (T):
   P(j) = e^(z_j/T) / Σe^(z_k/T)
   T kichik → "Ishonchli" (bir sinf dominant)
   T katta  → "Ishonchsiz" (teng taqsimot)
   → GPT da "temperature" parametri shu!
```

Softmax vs Sigmoid (ko'p sinf uchun):

```
Sigmoid har chiqishga mustaqil [0,1]:
  [0.9, 0.8, 0.7] → Yig'indi = 2.4 ≠ 1 ← Ehtimollik emas!
  Ko'p belgi bo'lishi mumkin holat (multi-label)

Softmax barcha chiqishni birga normallashtiradi:
  [0.659, 0.242, 0.099] → Yig'indi = 1.0 ✅
  Faqat bitta sinf (multi-class)
```

9. Aktivatsiya Funksiyalari Taqqoslash

```
Funksiya   | Diapazon     | Gradient   | Qachon
-----------|--------------|------------|--------------------
Sigmoid    | (0, 1)       | Yo'qoladi  | Binary chiqish
tanh       | (-1, 1)      | Yo'qoladi  | Eskirgan (LSTM)
ReLU       | [0, +∞)      | Yaxshi     | Yashirin qatlam
LeakyReLU  | (-∞, +∞)     | Yaxshi     | Dying ReLU bo'lsa
ELU        | (-α, +∞)     | Silliq     | Yashirin qatlam
Softmax    | (0,1), Σ=1   | —          | Ko'p sinf chiqish
Linear     | (-∞, +∞)     | Doimiy=1   | Regressiya chiqish

Amaliy qoida:
  Yashirin qatlam → ReLU (standart)
  Binary chiqish  → Sigmoid
  Ko'p sinf       → Softmax
  Regressiya      → Linear (aktivatsiyasiz)
```

---

10. Sklearn va Keras da Amaliyot

```python
# Sklearn — Sodda MLP:
from sklearn.neural_network import MLPClassifier

model = MLPClassifier(
    hidden_layer_sizes=(256, 128, 64),  # 3 yashirin qatlam
    activation="relu",                  # ReLU
    solver="adam",                      # Optimizer
    max_iter=300,
    random_state=42
)

# Keras — Chuqur tarmoq:
from tensorflow import keras

model = keras.Sequential([
    keras.layers.Dense(256, activation="relu",    input_shape=(784,)),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dense(64,  activation="relu"),
    keras.layers.Dense(10,  activation="softmax") # 10 sinf
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",  # Softmax uchun
    metrics=["accuracy"]
)
```

---

11. Xulosa

```
Biologik → Sun'iy:
  Dendrit  → Kirish (x)
  Sinaps   → Vazn (w)
  Soma     → Σwᵢxᵢ + b
  Aktivats.→ f(z)
  Akson    → Chiqish

Perceptron:
  ŷ = f(Wᵀx + b)
  XOR hal qila olmaydi → Ko'p qatlam kerak

Qatlamlar:
  Input  → Ma'lumot kirishi, hisoblash yo'q
  Hidden → Nochiziqli xususiyat o'rganish
  Output → Sinf/Qiymat chiqarish

Aktivatsiya funksiyalari:
  ReLU    → max(0,z)    → Zamonaviy standart, tez
  Sigmoid → 1/(1+e⁻ᶻ)  → Binary chiqish, [0,1]
  Softmax → e^z/Σe^z   → Ko'p sinf, Σ=1

Nima uchun aktivatsiya kerak:
  → Nochiziqlilik kiritadi
  → Aktivatsiyasiz → Chiziqli model (ko'p qatlam bekorga)
```

> 💡 **Eslab qol:** Neyron tarmoqning "chuqurligi" — qatlamlar soni, "kengligi" — har qatlamdagi neyronlar soni. **Chuqurlik** murakkab naqshlarni, **Kenglik** ko'p variantlarni ushlab oladi. Zamonaviy GPT modellari — 96 qatlam, 96 ta attention head — bu "chuqur va keng" tarmoqning eng ekstremal misoli!